

Vasic-Digital-Reusable-Module-Suite

Einmal bauen, überall wiederverwenden - eine Flotte kleiner, entkoppelter, unabhängig getesteter Go- und KMP-Module.

Zusammenfassung

Eine umfangreiche Sammlung generischer, wiederverwendbarer Module, veröffentlicht unter den Namespaces `digital.vasic.*` (Go) und Kotlin Multiplatform. Jedes Modul ist eigenständig, wird unabhängig getestet und versioniert und als Submodul mit gleichem Codebestand von größeren Produkten (Catalogizer, HelixAgent und der gesamten Flotte) genutzt. Diese Seite bündelt die vielen kleinen Hilfsmodule, die als einzelne Seiten nur störendes Rauschen wären.

Kurzbeschreibung

Eine kuratierte Sammlung entkoppelter `digital.vasic.*`-Module - Infrastruktur-Grundbausteine (Authentifizierung, Cache, Datenbank, Konfiguration, Observability), AI/Agenten-Bausteine (RAG, VectorDB, Embeddings, MCP, Agentic, Planung) und defensive LLM-Schutzmechanismen (RedTeam, Normalize) - ergänzt um einen Kotlin Multiplatform-Spiegelsatz. Jedes Modul ist generisch, getestet und wiederverwendbar.

Ausführliche Beschreibung

Die Organisation vasic-digital setzt auf eine einzige strukturelle Grundannahme: eine „Verfassung plus viele entkoppelte, wiederverwendbare Submodule“-Philosophie, bei der generische Funktionalität niemals zweimal geschrieben wird. Statt monolithischer Architekturen wird jede wiederverwendbare Komponente in ein eigenes kleines Modul ausgelagert - mit eigenem Repository, eigenen Tests und eigener Dokumentation - und strikt entkoppelt gehalten, sodass keine spezifischen

Anforderungen der Nutzer einfließen. Diese Seite fasst sie zusammen, weil jedes Modul für sich genommen bibliotheksähnlich wäre und als einzelne Produktseite nur störend wirken würde. Zusammengenommen bilden sie jedoch den eigentlichen Hebel der Organisation: ein internes technisches Asset, das „ein neues Produkt bauen“ in „bewährte Teile zusammenfügen“ verwandelt – und die konkrete Grundlage für die Behauptung, dass diese Flotte das Rad nicht neu erfindet, sondern ein einziges, sehr gutes Rad pflegt und überall einsetzt.

Die Sammlung umfasst drei Cluster. **Infrastruktur-Grundbausteine** (Go) stellen die Basisdienste bereit, die jede Anwendung benötigt: auth (JWT/bcrypt), cache (Redis/TTL), database (Migrationen, duale SQLite/PostgreSQL), config, middleware, observability (Prometheus/OpenTelemetry), ratelimiter, security, storage (S3/MinIO), streaming (WebSocket-Hub), eventbus, filesystem (mehrprotokollfähig), discovery/mdns, http3, recovery, concurrency, lazy und mehr. **AI/Agenten-Bausteine** (Go) bilden das Fundament für AI-Systeme: rag, vectordb, embeddings, memory, conversation (Komprimierung mit unendlichem Kontext, Event Sourcing), mcp (Model Context Protocol), toolschema, skillregistry, agentic (graphenbasierte Workflow-Orchestrierung), planning (HiPlan/MCTS/Tree-of-Thoughts), benchmark (SWE-bench/HumanEval/MMLU), llmops, selfimprove (Reward-Modellierung/RLHF) und toon (Token-Oriented Object Notation). **Defensive LLM-Schutzmechanismen** bieten Werkzeuge für adversariale Robustheit: RedTeam (YAML-gesteuerte adversariale Testfälle), Normalize (Kanonisierung adversarialer Eingaben). Ein paralleler **Kotlin Multiplatform**-Satz spiegelt Kernmodule (Auth-KMP, Database-KMP, Security-KMP, UI-Komponenten-KMP usw.) für plattformübergreifende Anwendungen.

Warum wir es entwickelt haben

Jedes Mal von Grund auf neue Produkte (Catalogizer, HelixAgent, Herald und weitere) zu entwickeln, ist ineffizient und führt zu Inkonsistenzen. Indem wir jede generische Komponente in ein entkoppeltes, getestetes Modul auslagern, verbreiten sich Fehlerbehebungen und Verbesserungen über die gesamte Produktpalette, und jedes neue Produkt setzt sich aus bewährten Bausteinen zusammen.

Warum es bahnbrechend ist

Es handelt sich im Grunde um eine private „Standardbibliothek“ für den Aufbau von AI-spezifischen Backends – jene Schicht, die die meisten Teams nie entwickeln, weil sie zu sehr damit beschäftigt sind, Authentifizierung, Caching und die RAG-

Infrastruktur zum fünften Mal neu zu erfinden. Hier existieren Infrastruktur-Primitives, AI-Bausteine und defensive LLM-Sicherheitsvorkehrungen als sofort einsatzbereite, unabhängig getestete Module. Das ermöglicht es einem kleinen Team, produktionsreife Systeme in einem Tempo zu liefern, das normalerweise ein viel größeres Team erfordern würde – und das ohne die sonst übliche Duplikationsschuld.

Was innovativ ist

- **Fleet-weite Entkopplungsdisziplin (CONST-051):** Submodule werden als gleichwertige Codebasen behandelt, ohne spezifische Abhängigkeiten von Nutzern.
- **Eine dedizierte AI-Primitiv-Schicht (RAG, VectorDB, Embeddings, MCP, ToolSchema, Agentic, Planning, LLMOps)** als wiederverwendbare Module.
- **Ein Cluster defensiver LLM-Sicherheitsvorkehrungen (RedTeam, Normalize)** für robusten Schutz gegen Angriffe.
- **Parallele Go- und Kotlin Multiplattform-Modulsets**, die dieselben Konventionen teilen.

Herausforderungen & Lösungen

- **Vermeidung von Kopplungsverfall bei Dutzenden Modulen:** gelöst durch den Entkopplungsvertrag der Verfassung und die Laufzeit-Injektion nutzerspezifischer Anpassungen.
- **Konsistenz und Testabdeckung vieler Module:** gelöst durch gemeinsame Konventionen (pro Modul: Tests/ Dokumentation/Herausforderungen) und das HelixConstitution-Governance-Rückgrat.
- **Plattformübergreifende Reichweite:** gelöst durch ein Kotlin Multiplattform-Spiegelmodul der Kernkomponenten.

Technologie-Stack (Warum + Wie)

- **Go** – die Mehrheit der Module (digital.vasic.*).
- **Kotlin Multiplatform** – plattformübergreifende Spiegelmodule (Auth/Database/Security/UI/Concurrency/RateLimiter-KMP).
- **Redis / PostgreSQL / SQLite** – Cache-, Datenbank- und Speicherprimitive.
- **Prometheus / OpenTelemetry** – Observability-Modul.
- **WebSocket / HTTP/3 (quic-go) / mDNS** – Netzwerkmodule.
- **Vector DB / embeddings / RAG / MCP** – AI-Primitivmodule.
- **YAML** – RedTeam-Angriffsszenarien und Konfiguration.

UNGEPRÜFT / IN ARBEIT: Mehrere Organisations-Repos sind als „SCAFFOLD / WIP“ gekennzeichnet (z. B. PliniusCommon, I-LLM, HyperTune, AutoTemp, Veritas, Ouroborous, Claritas, LeakHub, GandalfSolutions). Diese sind als frühe Entwicklungsstadien/Scaffolding zu betrachten, nicht als ausgelieferte Komponenten.